

FINAL PROJECT PART 2

Indexing and Evaluation

Laura NARANJO & Laura PEÑASALVER & Aurora PUJOLS

November 2, 2025

TAG: IRWA-2025-part-2
GitHub: <https://github.com/aurorapujols/irwa-search-engine>

We separate this part 2 of the project in two parts:

- a. **Indexing (1)** in which we load the corpus (product's articles), create the **inverted index**, the **ranking system** to rank our documents, and the retrieval system (**search engine**).
- b. **Evaluation (2)**: in which we **evaluate our retrieval system** for the specified queries, and discuss the limitations and possible improvements for our system.

1 PART 1: Indexing

In this section we explain how we decided to create the index, how we rank the products, and how the search engine works with the index and the ranking integrated.

AI use

INDEX: It gave us ideas on how to incorporate the product fields into the index to use them later in the ranking. Mainly that we could create sub-indexes inside the index, or that we could include the field in the posting jointly with the product ID and the positions.

RANKING: In this part, we used ChatGPT to give us ideas on how to weight the different fields when computing the tf_idf score. We first thought to include the fields in the inverted document frequency, but later we had trouble computing the score, so we asked ChatGPT for ideas to integrate the weights into it. Basically, we could try to do the integration with the inverted document frequency, or do it with the term frequency. We chose the latter because it made sense conceptually.

CODE: Most of the code for this part is exclusively made by us, with the help of the code we already did in Practice Session 1. We used ChatGPT to fix an error when computing the rank.

DEBUGGING: We also used AI tools for debugging, especially when dealing with compilation errors and JSON formatting.

We created a file named `index_tf_idf.py` in which we defined all the functions related to the index creation and document ranking. This file also calls this functions:

```
# (1) Load the corpus
print("\033[34mLoading corpus...\033[0m")
json_path = os.getenv("DATA_FILE_PATH")
corpus = corpus_df_loading(json_path)

# (2) Compute (load if already stored) the index and save it together with the additional
# information
index, index2title, tf, df, idf = get_index_and_metrics(corpus)

# (3) User Interface to query the system seamlessly
```

```

print("\033[33m\nClothing Articles Search Engine\033[0m")
exit = False
while(not exit):
    query = input("\033[33m\nEnter search terms: \033[0m")
    scored_products = engine_search(query, index, tf, idf, weights)
    if scored_products != None:
        print(f"\033[32mProducts found (10/{len(scored_products)}):\033[0m")
        for idx, (score, pid) in enumerate(scored_products):
            if(idx >= 10):
                break
            print(f"{idx+1}. [tf-idf={score:.3f}] {index2title[pid]}")
    else:
        print("\033[32mNo results found!\033[0m")

    answer = input("\033[33mWould you like to do another search[Y/n]: \033[0m").strip().
        lower()
    exit = answer in ("N", "n", "no", "No", "NO")

```

We loaded the corpus (1) before creating the index, and we used the function `load_corpus` provided in the repository in the function `load_corpus_from_json`.

1.1 Index Creation

To create the index, we created a function named `create_index_tf_idf`, where we also compute the term frequency, document frequency, and inverted document frequency. As we explain in the Jupyter notebook `indexing_and_evaluation.ipynb`, we wanted to develop the index with sub-indexes corresponding to the fields like we explained in **Part 1** to give different levels of importance to a term appearing in one field or in another.

The first step to calculate this index was to preprocess the content in all fields as we described in **Part 1**. Once all fields were pre-processed, we proceeded to generate the index going through all the words in the document, similar to the procedure done in Practice Session 1. Our procedure only differentiates in how to handle the fields: the inverted index we generated has sublevels indicating the field in which you can find the term, and the positions stored are relative to said field. The structure of the index is described and exemplified in the Jupyter notebook `indexing_and_evaluation.ipynb`, and you will see later that the term frequency was also stored in the same way.

The document frequency and the inverted document frequency remain the same as in the classical approach. Counting the document frequency became more complicated with our approach; because we went through each field in a product, there was the possibility of seeing a term more than once if it appeared in two fields or more. We had to use an extra structure across fields that kept track of the terms we had seen to include them only once.

1.1.1 Term frequency per Field

Computing the term frequency requires normalisation over the total number of terms, in this case of the product. We needed to adapt the standard version of the term frequency to our nested index. For this, we decided to consider the term frequency per field instead of per product. This is why our `tf` structure is indexed by term, field, and document in this order.

1.2 Document Filtering and Ranking

The reasoning explained in this section is shown in detail in the Jupyter notebook `indexing_and_evaluation.ipynb`, including code pieces and explicit examples of representations.

Our goal was to rank documents in which all the words in the user's query appear at least once. Based on the work done in Practice Session 1, we decided to filter the documents and reduce them to those that contain the query words before ranking them. This way, we avoid computing the rank score for products we know will be useless to the user in the end. The final list of products to rank is the intersection of all the product that contain at least one of the query words. We created a function named `filter` that does this. The function receives the user query and the generated index, and returns the query terms separated as well as the list of relevant documents.

To rank our documents, we used an adapted version of the `tf-idf` score that includes the product fields. To do this, we created a function called `rank_tf_idf` where we compute the relevance score of each document with respect to the query. This is the most creative part of our code: we consider that each product field has

its own behaviour and decided to weight the presence of a term differently depending on the field it appears on. Basically, we do not measure the term frequency by counting the number of appearances in a document. Instead, we do a weighted average of the number appearances in the different product fields. Our current weight configuration is as follows:

- Term appearing in **title** or **description**: 1.0
- Term appearing in **brand**: 0.5
- Term appearing in **category**: 0.3
- Term appearing in **subcategory**: 0.4
- Term appearing in **seller**, **product**, or **details**: 0.2

These have been chosen based on the features we consider the most important. It is possible to find an optimal weight configuration, for example by evaluating multiple configurations and evaluating the performance, or by treating the fields as features and using methods like Learning to Rank. We leave it as possible future work.

1.3 Search Engine Integration

With the index created and the function to rank the documents completed, we were ready to integrate the two into the search engine and make it capable of interacting with the user. For this, we created a function named `search_engine` that processes a given query (i.e. builds the query terms and filters the documents), ranks the relevant documents, and gives back the list of documents sorted in order of most relevant to least relevant. In this step, we are still missing the User Interaction part.

We are going to have the actual program, in this case the python module, interact with the user. There is a small interface that asks the user for queries and communicates these queries to the search engine. The interface can handle multiple queries in continuous mode, meaning the user can ask a new query after getting the results of a previous one.

2 PART 2: Evaluation

AI use

DEBUGGING: For this part, we only used AI tools for debugging and helping with the plots.

As explained in the Jupyter notebook `indexing_and_evaluation.ipynb`, we created a csv file with the results of the queries in the same format as the one given to us (`validation_labels.csv`) and manually added the labels to the relevant (label=1) and non-relevant (label=0) documents for all results.

From that, we applied the functions to compute the evaluation metrics to the new csv file. We used $k = 14$ because the query with the least results gave only 14 ranked products. The results are in Tables 1 and 2. Recall that our queries are the following:

- **Query 1:** "western leather jacket men"
- **Query 2:** "cotton innerwear man"
- **Query 3:** "yellow black t-shirt women xl"
- **Query 4:** "casual comfortable blue trousers women"
- **Query 5:** "breathable sports clothes winter"

Additionally, in Figures 1, 2, 3, 4, and 5, we plotted the precision-recall curves for each query to see the performance for all the retrieved documents, and we included other metrics' representation as well.

	P@K	R@K	Average P@K	F1@K	NDCG@K
Query 1	0.929	0.812	0.929	0.867	0.953
Query 2	0.929	0.867	0.929	0.897	0.952
Query 3	0.286	1.000	0.286	0.444	0.905
Query 4	0.429	0.857	0.429	0.571	0.841
Query 5	1.000	1.000	1.000	1.000	1.000

Table 1: Table of evaluation metrics for query-specific metrics

MAP	MRR
0.929	1.000

Table 2: Mean Reciprocal Rank (MRR) and Mean Average Precision of the system.

2.1 Evaluation of system for our queries

As all the formulas to compute the evaluation metrics are on the jupyter notebook, here we limit to analyze the meaning of each metric and the results:

- **PrecisionK**: it measures the proportion of **relevant documents among the top K** retrieved documents. If PK is high, it means that the system is retrieving mostly relevant results early in the ranking. In our case, most queries (except 3 and 4) are showing **high precision**, which means that their rankings gave relevant documents in most of the top k ranks. In query 3 and 4, a lot **less relevant documents are retrieved** early in the rank. In figures 3 and 4 we can see how the precision drops considerably at high recall (up to 0.24 in query 3, and to 0.14 in query 4) but it stays above 0.6 for the rest of queries (figures 1, 2, and 5).
- **RecallK**: it measures the proportion of relevant documents that are retrieved in the top K . If a system has high recall, it means that it finds most of the relevant items. In our case, we consider all relevant items as the total amount we found in the result of the queries. We can see in table 1 that all recall values are high (above 0.810). This because within the first 14 queries, almost all relevant documents were retrieved in all queries (or all if recall=1). However, if we knew exactly for each query all the relevant documents in the corpus, maybe the number of relevant items would be higher and then these values would decrease. Because we only consider relevant documents among the retrieved ones, at the graphs of precision-recall curves, all queries end with recall 1.
- **Average Precision@K**: it averages precision across different cutoff points up to K . Its main aim is to smooth precision fluctuations across ranks. In our case, the averages coincide with the values for P@K. It still shows that all queries except query 3 and 4, have very good ranked solutions.
- **F-score@K**: it is the harmonic mean of precision and recall at K . This measure balances precision and recall so that it penalizes systems that perform well at one K , but fail in another. There's a lot of balance between precision and recall in queries 1, 2, and 5 (F-score above 0.860), but in the case of query 3 and 4, the values are much lower (0.444 and 0.571, respectively). This shows that, as seen in figures 3, and 4, the curves have sudden and big drops, going from performing relatively well to very bad. In other words, the first documents are ranked well, but at some k the documents are not relevant.
- **Normalized Discounted Cumulative Gain@K**: it measures the ranking quality by rewarding relevant documents that appear in higher rank positions and penalizes the ones at lower rank positions. In our queries, the scores at $k=14$ are quite good except for query 4. This is because, even though there are less relevant documents ranked in query 3, they all are in high ranking positions (as well as queries 1, 2, and 5). But in the case of query 4, more relevant documents are ranked after non relevant ones, decreasing the NDCG@ k .
- **Mean Reciprocal Rank**: it measures the average of reciprocal ranks of the first relevant result. It is trying to evaluate how quickly the system retrieves the relevant items (quickly as in the highest ranking positions). In our case, the result is 1.000, which is an excellent result. This means that, for all queries, the first item retrieved was a relevant document.
- **Mean Average Precision**: it averages the precision across all relevant documents for all queries. In our case, this value is 0.929. The result is very good meaning that the overall precision of the system is close

to excellent.

Other insights we get are:

- Inn query 5 (figure 5, and the last row of table 1), performs for this query as a perfect system for our requirements and relevance labels. All values are 1.000, and the precision-recall curve is a horizontal line at precision 1.000.
- We only considered 5 queries, and we do not know if these would be actual queries the user would use in our system. Meaning that in a real use case, maybe these results are not accurate at all.
- We also need to think about the limited amount of documents we select for the ranking as we only rank those documents containing ALL query terms. By doing this filtering, maybe for some queries there are very few relevant documents and although we retrieve all of them, there are many more that are not relevant. Then for a higher K than the number of relevant documents, the metrics will decrease considerable but maybe the ranking was perfect from the documents we ranked.

2.2 Limitations and Improvements

- a. **Retrieval Precision and Field Type Handling:** now, we are using weights for each field when computing the tf-idf. But these weights were assigned based on our fast criterion in a very short evaluation of the fields' importance. But it is all empirical. This might cause the ranking to be not as good as having well calculated weights per field, or even worse than having the same weight for every field.

FIX: we are already implementing the field distinction between different fields but the weights might not be appropriate. We could improve their values by using relevance evaluated data and having several candidate weights, and keeping the ones that optimize the performance measures (e.g. using GridSearch).

- b. **Ranking Quality:** specifically for queries with results like query 3 and 4, the NDCG and F1 scores drop when relevant items are ranked lower.

FIX: use better ranking models like BM25, or even learning to rank models to capture better the relevance beyond term frequency.

- c. **Query Formulation:** the queries we are using are very specific knowing the term frequencies, and the vocabulary in the collection. But for an external user, new queries might be completely different or badly formulated for how we designed our system.

FIX: use query enhancing and query suggestion knowing the vocabulary in the corpus. If the user is typing a query and we suggest similar words to the ones in the dictionary and they are actually interesting for the user, they will help retrieved better products. Also, using synonyms to enhance the query might also improve the ranking.

- d. **Limited documents:** by performing the filtering before the ranking with only the documents with ALL query terms, we are maybe omitting products that are relevant but do not contain all the specific terms.

FIX: use all documents in the corpus that have at least one query term to still limit the amount of documents to rank, but perform the ranking for possibly omitted products in our implementation. Counterpoint: the processing would be much slower.

- e. **Extra Filtering:** by using the current search engine, we are not using the numerical values of the products (price, discounts, etc.).

FIX: if we add a special selection for the user to filter in or out documents based on these characteristics, the ranking might be more accurate with more relevant documents not only chosen by the query terms.

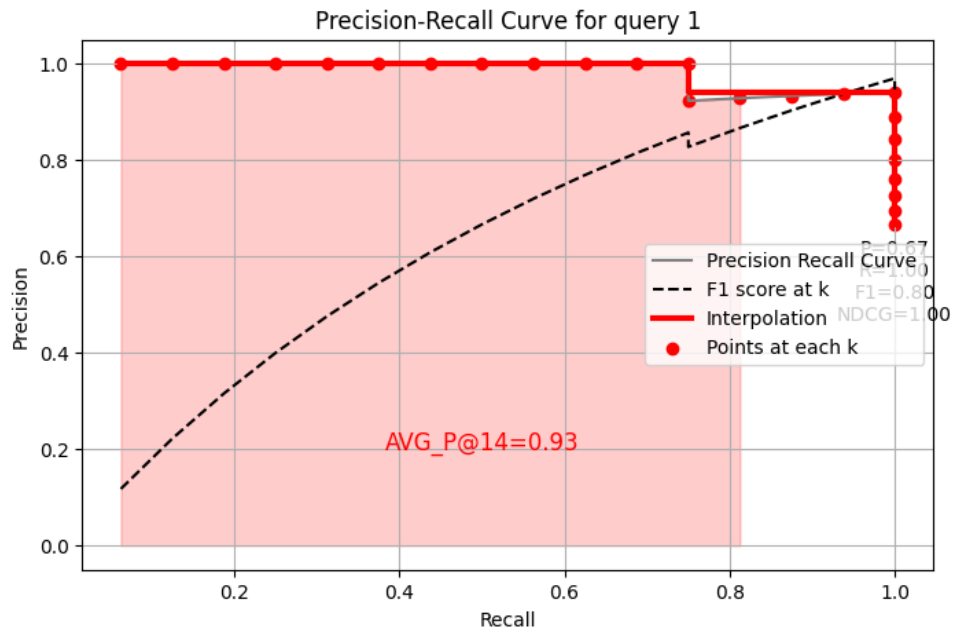


Figure 1: Query 1: "western leather jacket men"

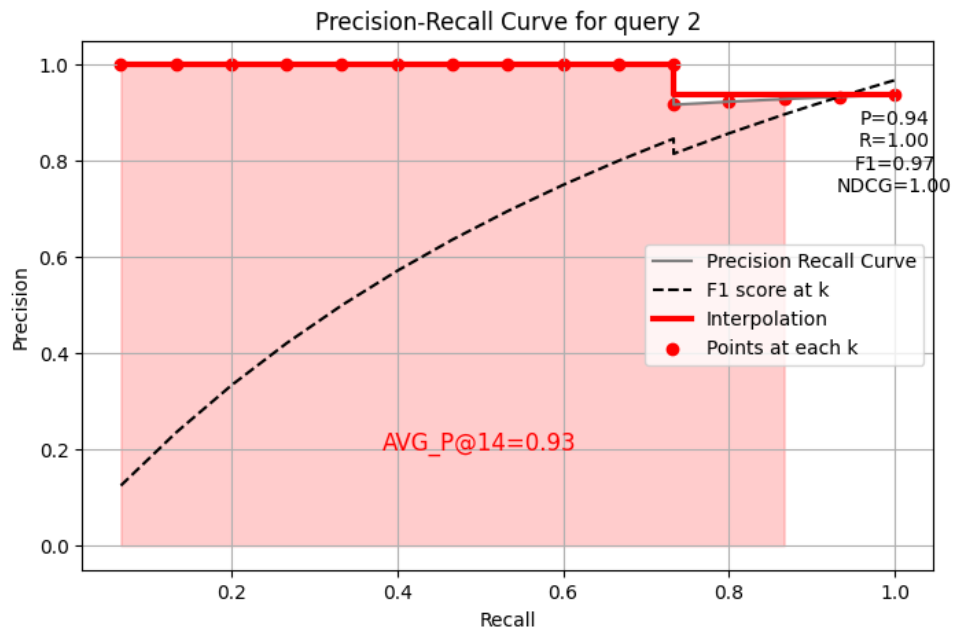


Figure 2: Query 2: "cotton innerwear men"

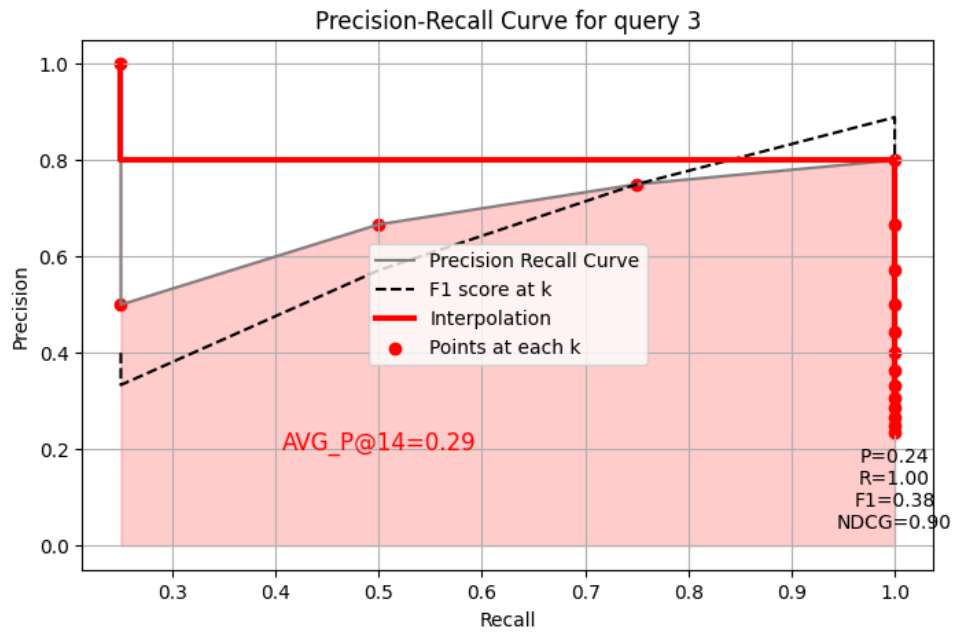


Figure 3: Query 3: "yellow black t-shirt women xl"

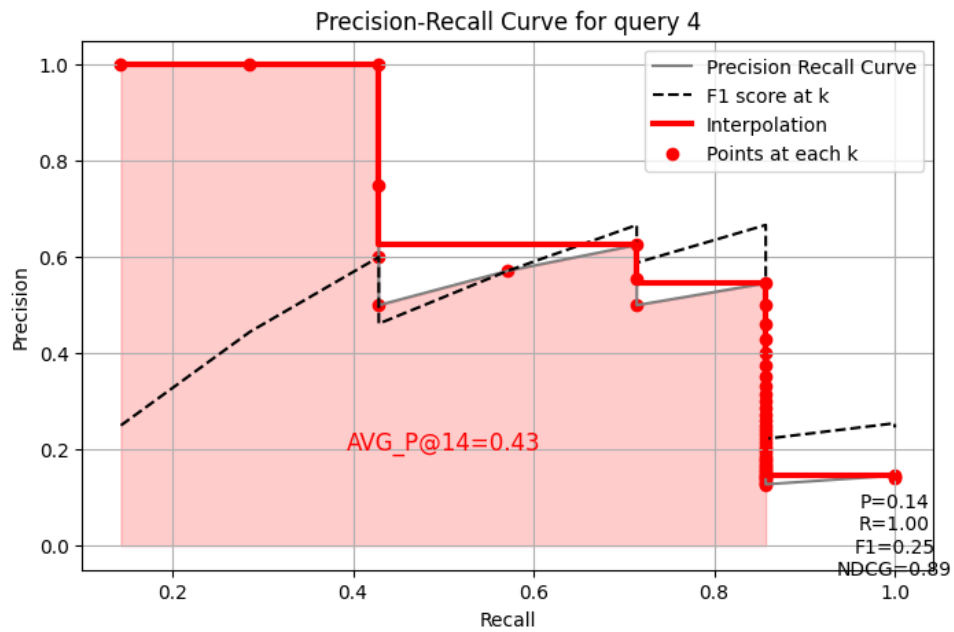


Figure 4: Query 4: "casual comfortable blue trousers women"

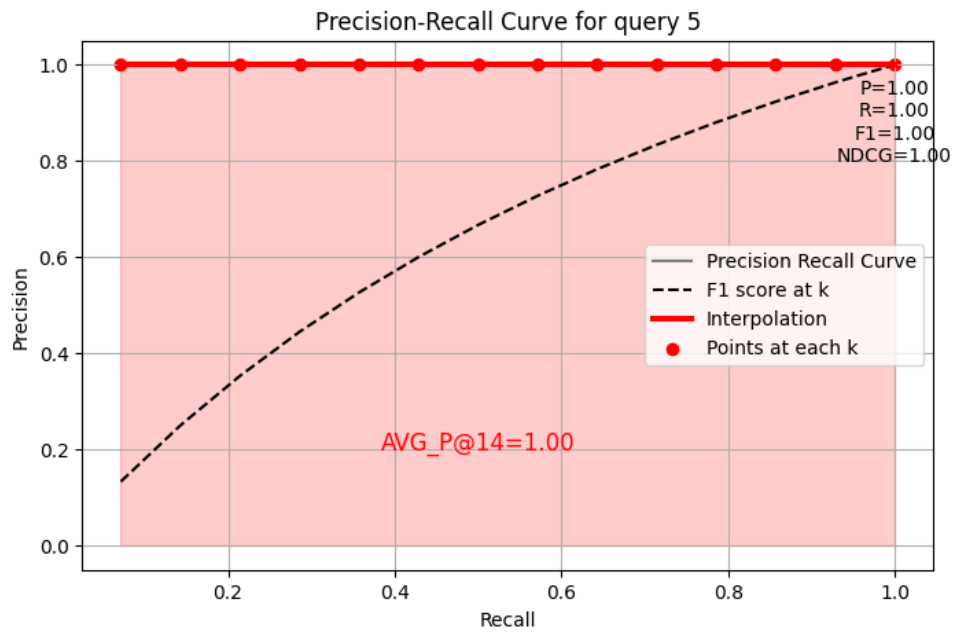


Figure 5: Query 5: "breathable sports clothes winter"